

Syllabus

Unit I	Introduction to Databases and DBMS	7 hrs
	Database Basics - Definition, importance, and applications of databases. - Data vs. Information, Database vs. File System. - Characteristics of databases. Database Management Systems - DBMS architecture: Levels of abstraction (physical, logical, and view levels). - Types of DBMS: Relational, NoSQL, Object-Oriented. - Database models: Hierarchical, Network, and Relational models. Database Environment - Role of database administrators (DBAs) and database designers. - Overview of database users and their roles.	
Unit II	Relational Database Design and Normalization	8 hrs
	Relational Model Concepts - Attributes, tuples, relations, and schema. - Keys: Primary, candidate, foreign, and superkeys. Relational Algebra and Operations - Basic operations: Selection, projection, union, intersection, difference, and Cartesian product. - Advanced operations: Joins, division, aggregation, and grouping. Database Design Process - Entity-Relationship (ER) modeling: Entities, attributes, relationships, and ER diagrams. - Conversion of ER diagrams to relational schema. Normalization - Functional dependencies. - Normal forms: 1NF, 2NF, 3NF, and BCNF (with examples). - Denormalization and its applications.	
Unit III	SQL and Query Optimization	8 hrs
	Structured Query Language (SQL) - Basics of SQL: Data definition (CREATE, ALTER, DROP) and data manipulation (INSERT, UPDATE, DELETE). - Querying data: SELECT, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT. - Joins: Inner, outer, self-joins, and cross-joins. - Subqueries and set operations. Advanced SQL Features	

	○ Views, Indexes, and Triggers. ○ Stored Procedures and Functions. ○ Role of constraints (NOT NULL, UNIQUE, CHECK, DEFAULT). 3. Query Optimization ○ Query execution plans. ○ Optimization strategies: Indexing and query rewriting.	
Unit IV	Transactions, Concurrency, and Recovery	8hrs
	1. Transaction Management ○ ACID properties: Atomicity, Consistency, Isolation, Durability. ○ Transaction states and lifecycle. 2. Concurrency Control ○ Problems in concurrent transactions: Lost updates, uncommitted data, and inconsistent analysis. ○ Lock-based protocols: Shared, exclusive locks, and two phase locking. ○ Deadlock handling in DBMS. 3. Database Recovery ○ Types of failures: System crash, media failure. ○ Recovery techniques: Log-based recovery, shadow paging. ○ Checkpointing and its significance.	
Unit V	Emerging Trends in Databases	8 hrs
	1. NoSQL Databases ○ Types of NoSQL databases: Key-value stores, document databases, column-family stores, graph databases. ○ Use cases and advantages of NoSQL over relational databases. ○ Introduction to popular NoSQL systems (MongoDB, Cassandra, Redis). 2. Big Data and Distributed Databases ○ Basics of distributed databases and CAP theorem.	

Unit 3

SQL and Query Optimization

A conceptual evaluation strategy is a way to evaluate the query that is intended to be easy to understand, rather than efficient. A DBMS would typically execute a query in a different and more efficient way.

SQL stands for **Structured Query Language**.

It is a **standard programming language** used to **manage and manipulate databases**.

Common SQL Data Types

1. Numeric Data Types

- **INT / INTEGER**: Whole numbers (e.g., 1, 100, -50)
- **SMALLINT**: Smaller range of integers
- **BIGINT**: Larger range of integers
- **DECIMAL(p, s) / NUMERIC(p, s)**: Fixed precision numbers (p = total digits, s = digits after decimal)
- **FLOAT / REAL / DOUBLE**: Approximate floating-point numbers

2. Character/String Data Types

- **CHAR(n)**: Fixed-length string of length n (padded with spaces if shorter)
- **VARCHAR(n)**: Variable-length string, max length n
- **TEXT**: Large text data (size limit depends on DBMS)

3. Date and Time Data Types

- **DATE**: Stores date (year, month, day)
- **TIME**: Stores time (hour, minute, second)
- **DATETIME** or **TIMESTAMP**: Stores date and time together
- **YEAR**: Stores year only (some DBMS)

4. Boolean Data Type

- **BOOLEAN** or **BOOL**: Stores TRUE or FALSE values (implementation varies by DBMS; often stored as tinyint 1 or bit)

5. Binary Data Types

- **BINARY(n)**: Fixed-length binary data
- **VARBINARY(n)**: Variable-length binary data
- **BLOB**: Large binary objects (images, files, etc.)

Basic Schema Definition: the **basics of SQL**, divided into two main parts-

1. DDL
2. DML

1. **DML (Data Manipulating Language)**: These commands define or modify the structure of database objects like tables.

a) CREATE — Create a table

```
CREATE TABLE Students (
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(100),
  Year INT,
  Major VARCHAR(50)
);
```

b) ALTER — Modify an existing table

- Add a new column:

```
ALTER TABLE Students
ADD Email VARCHAR(100);
```

- Modify a column datatype (syntax varies per DBMS):

```
ALTER TABLE Students
ALTER COLUMN Major VARCHAR(100);
```

```
Drop a column:
ALTER TABLE Students
DROP COLUMN Email;
```

c) DROP — Delete a table or database object

```
DROP TABLE Students;
```

2. Data Manipulation Language (DML)

These commands manipulate the data stored in tables.

a) INSERT — Add new rows

```
INSERT INTO Students (StudentID, StudentName, Year, Major)
VALUES (1, 'Alice', 2, 'Computer Science');
```

b) UPDATE — Modify existing data

```
UPDATE Students
SET Major = 'Mathematics'
WHERE StudentID = 1;
```

c) DELETE — Remove rows

```
DELETE FROM Students
WHERE StudentID = 1;
```

3. Querying Data (SELECT)**a) SELECT — Retrieve data**

```
SELECT StudentName, Major
FROM Students;
```

b) WHERE — Filter rows based on condition(s)

```
SELECT StudentName, Major
FROM Students
WHERE Year = 2;
```

c) GROUP BY — Aggregate rows with common values

Example: Find the number of students in each major:

```
SELECT Major, COUNT(*) AS StudentCount
FROM Students
GROUP BY Major;
```

Bonus: Combining WHERE and GROUP BY

Find the count of second-year students in each major:

```
SELECT Major, COUNT(*) AS StudentCount
```

```
FROM Students
WHERE Year = 2
GROUP BY Major;
```

JOIN in SQL: A **JOIN** is used to combine rows from **two or more tables** based on a related column between them.

Types of JOINS

1. INNER JOIN

- Returns only the rows where there is a match in **both tables**.

```
SELECT A.column1, B.column2
FROM TableA A
INNER JOIN TableB B ON A.common_column = B.common_column;
```

2. LEFT JOIN (or LEFT OUTER JOIN)

- Returns **all rows from the left table**, and matching rows from the right table.
- If no match, right table columns are NULL.

```
SELECT A.column1, B.column2
FROM TableA A
LEFT JOIN TableB B ON A.common_column = B.common_column;
```

3. RIGHT JOIN (or RIGHT OUTER JOIN)

- Returns **all rows from the right table**, and matching rows from the left table.
- If no match, left table columns are NULL.

```
SELECT A.column1, B.column2
FROM TableA A
RIGHT JOIN TableB B ON A.common_column = B.common_column;
```

4. FULL JOIN (or FULL OUTER JOIN)

- Returns all rows when there is a match in **either** left or right table.
- Non-matching rows will have NULLs on the side that doesn't match.

```
SELECT A.column1, B.column2
FROM TableA A
FULL OUTER JOIN TableB B ON A.common_column = B.common_column;
```

5. CROSS JOIN

- Returns the **Cartesian product** of both tables (all combinations).
- Be careful: this can create very large result sets!

```
SELECT A.column1, B.column2
FROM TableA A
CROSS JOIN TableB B;
```

Subqueries and Set Operations in SQL

1. Subqueries (Nested Queries)

A **subquery** is a query inside another query. It allows you to use the result of one query as input for another.

Types of Subqueries:

- **Scalar subquery:** Returns a single value
- **Column subquery:** Returns a single column (multiple rows)
- **Row subquery:** Returns a single row with multiple columns
- **Table subquery:** Returns a table (multiple rows and columns)

Subquery in WHERE clause

Find students who enrolled in the course with the highest credits:

```
SELECT StudentID, StudentName
FROM Students
WHERE StudentID IN (
    SELECT StudentID
    FROM Enrollments
    WHERE CourseID = (
        SELECT CourseID
        FROM Courses
        ORDER BY Credits DESC
        LIMIT 1
    )
);
```

Example 2: Subquery in SELECT clause

Find each student's total credits enrolled:

```
SELECT
    s.StudentID,
    s.StudentName,
    (SELECT SUM(c.Credits)
     FROM Enrollments e
     JOIN Courses c ON e.CourseID = c.CourseID
     WHERE e.StudentID = s.StudentID) AS TotalCredits
FROM Students s;
```

2. Set Operations

Set operations combine results from **two or more SELECT queries**.

Types of Set Operations:

a) UNION

- Combines results of two queries and **removes duplicates**.

```
SELECT StudentName FROM Students WHERE Year = 2
UNION
SELECT StudentName FROM Students WHERE Major = 'Mathematics';
```

b) UNION ALL

- Combines results and **includes duplicates**.

```
SELECT StudentName FROM Students WHERE Year = 2
UNION ALL
SELECT StudentName FROM Students WHERE Major = 'Mathematics';
```

c) INTERSECT

- Returns rows **common to both queries** (available in some DBMS like PostgreSQL, SQL Server).

```
SELECT StudentName FROM Students WHERE Year = 2
INTERSECT
SELECT StudentName FROM Students WHERE Major = 'Mathematics';
```

d) EXCEPT or MINUS

- Returns rows in the first query that are **not in the second query**.

```
-- In SQL Server (EXCEPT)
SELECT StudentName FROM Students WHERE Year = 2
EXCEPT
SELECT StudentName FROM Students WHERE Major = 'Mathematics';

-- In Oracle (MINUS)
SELECT StudentName FROM Students WHERE Year = 2
MINUS
SELECT StudentName FROM Students WHERE Major = 'Mathematics';
```

In SQL, **aggregate functions** are used to perform calculations on multiple rows of a table's column and return a single value. They are commonly used with the `GROUP BY` clause but can also be used without it to get overall summaries.

Common Aggregate Functions in SQL:

1. **COUNT ()**: Counts the number of rows or non-NULL values in a column.

```
SELECT COUNT(*) FROM employees; -- Counts all rows
SELECT COUNT(salary) FROM employees; -- Counts non-NULL salaries
```

2. **SUM ()**: Adds up all the values in a numeric column.

```
SELECT SUM(salary) FROM employees;
```

3. **AVG ()**: Calculates the average (mean) of a numeric column.

```
SELECT AVG(salary) FROM employees;
```

4. **MIN ()**: Finds the minimum value in a column.

```
SELECT MIN(salary) FROM employees;
```

5. **MAX ()**: Finds the maximum value in a column.

```
SELECT MAX(salary) FROM employees;
```

Views

A **view** is a **virtual table** based on the result of a `SELECT` query. It doesn't store data itself, but presents data from one or more tables.

```
CREATE VIEW view_name AS SELECT column1, column2 FROM table_name WHERE condition;
```

Example:

```
CREATE VIEW active_employees AS SELECT id, name, department
FROM employees WHERE status = 'active';
```

Now, you can do:

```
SELECT * FROM active_employees;
```

2. Indexes

An **index** is used to **speed up the retrieval** of rows from a table. It's like an optimized lookup for specific columns.

Syntax:

```
CREATE INDEX index_name ON table_name (column1, column2, ...);
```

Example:

```
CREATE INDEX idx_lastname ON employees (last_name);
```

TRIGGERS

A trigger is a procedure that is automatically invoked by the DBMS in response to specified changes to the database, and is typically specified by the DBA. A database that has a set of associated triggers is called an active database. A trigger description contains three parts:

Event: A change to the database that activates the trigger.

Condition: A query or test that is run when the trigger is activated.

Action: A procedure that is executed when the trigger is activated and its condition is true.

A trigger can be thought of as a ‘daemon’ that monitors a database, and is executed when the database is modified in a way that matches the *event* specification. An insert, delete or update statement could activate a trigger, regardless of which user or application invoked the activating statement; users may not even be aware that a trigger was executed as a side effect of their program.

Examples of Triggers in SQL

The examples shown in Figure 5.19, written using Oracle 7 Server syntax for defining triggers, illustrate the basic concepts behind triggers. (The SQL:1999 syntax for these triggers is similar; we will see an example using SQL:1999 syntax shortly.) The trigger called *init count* initializes a counter variable before every execution of an INSERT statement that adds tuples to the Students relation. The trigger called *incr count*

increments the counter for each inserted tuple that satisfies the condition *age* < 18.

```
CREATE TRIGGER initcount BEFORE INSERT ON Students /* Event */
DECLARE
    count INTEGER;
BEGIN
    count := 0;
END

CREATE TRIGGER incr count AFTER INSERT ON Students /* Event */
WHEN (new.age < 18) /* Condition; 'new' is just-inserted tuple */
FOR EACH ROW
BEGIN
    /* Action; a procedure in Oracle's PL/SQL syntax */
    count := count + 1;
END
```

Figure 5.19

Examples Illustrating Triggers